

Comparing Methods for Finding Roots of Polynomials

By

Danny Halovanic



West Chester University of Pennsylvania

May 7, 2025

Contents

1	Introduction	2
2	Literature Review	2
2.1	Real-Life Application	3
3	Methodology	4
3.1	Bisection Method	4
3.2	Newton's Method	4
3.3	Cubic Formula	5
4	Analysis	5
4.1	Quadratic Case	5
4.2	Cubic Case	7
5	Conclusion	8
6	Further Research	8

1 Introduction

The search for roots of a polynomial has been a question for many centuries. Euclid used geometry to solve quadratic equations. In China and India, mathematicians developed various algorithms to solve polynomial equations. For polynomials of degree 4 or less, exact algebraic formulas have been discovered. Although the algebraic solutions are precise when calculating by hand, the natural issues arise when calculated via a calculator or a computer: Can these algebraic formulas still yield accurate numerical results?

Several hundred years ago, exact algebraic forms for cubic and quartic polynomials were discovered. However, Evariste Galois proved using Galois theory that there do not exist exact general algebraic forms for polynomials of degree 5 or greater. Additionally, despite the exact general formulas for polynomials of degree 4 or less, the general formulas are verbose and messy. This is where numerical estimations come into play.

In computational mathematics, various root-finding algorithms have been used to solve roots numerically when algebraic formulas are not possible or convenient. The most widely used root-finding algorithms include the bisection method and Newton's method. The bisection method is guaranteed to converge within a correct interval, while Newton's method converges much more quickly but requires an accurate initial guess and is sensitive to function behavior.

2 Literature Review

R.D. Middlebrook introduces a new quadratic root-finding algebraic formula known as the **low-entropy method**. The term *low-entropy* refers to expressions whose origin of terms is transparent, making the expression more computationally stable. [3] The traditional quadratic formula is defined as:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (1)$$

While frequently used, the quadratic formula introduces numerical errors involving the discriminant:

$$\sqrt{b^2 - 4ac} \quad (2)$$

When the coefficients a and c are small relative to b , the subtraction within the discriminant is flawed due to truncation or cancellation errors. In this scenario, the discriminant is approximated as:

$$\sqrt{b^2 - 4ac} \approx \sqrt{b^2} = b \quad (3)$$

Using this poor approximation in the quadratic formula results in:

$$x \approx \frac{-b + b}{2a} = \frac{0}{2a} = 0 \quad (4)$$

To resolve the precision loss issue, Middlebrook modifies the quadratic formula by introducing two parameters, Q and F , defined as:

$$Q = \frac{\sqrt{(c/a)}}{b/a}, \quad F = \frac{1}{2} + \frac{1}{2}\sqrt{1 - 4Q^2} \quad (5)$$

The parameters help to maintain the structure of the original formula while preventing numerical instability.

Using the parameters, the two roots are expressed in a more numerical stable form:

$$x_1 = -\frac{c}{b} \frac{1}{F}, \quad x_2 = -\frac{b}{a} F \quad (6)$$

The low-entropy form of a quadratic polynomial provides numerical accuracy in addition to providing a physical interpretation in real-life applications.

2.1 Real-Life Application

RD Middlebrook devised the low-entropy form of the quadratic equation due to its usefulness in engineering, especially pertaining to electronic circuits. The traditional quadratic formula can be considered unstable in certain real-life applications. Middlebrook aimed to simplify the original formula while maintaining numerical accuracy.

In his paper, Middlebrook applied this technique in a real-world scenario involving a second-order electrical circuit. He started with the voltage transfer function, or the voltage gain as defined is as the equation:

$$A = v_1/v_2 \quad (7)$$

where v_1 represents the output voltage and v_2 the input voltage, measured in volts. The ratio in (7) describes how the signal is modified throughout the system.

The frequency response of the system is calculated by two poles, the roots of a quadratic equation. These poles are defined as:

$$\omega_1 = Q\omega_0 \frac{1}{F} \quad \omega_2 = \frac{\omega_0}{Q} F \quad (8)$$

where ω_0 is the natural resonant frequency of the circuit and F is defined the same as (5).

Middlebrook demonstrates then that following ratio exists between the two poles:

$$\frac{\omega_1}{\omega_2} = \frac{Q^2}{F^2} \quad (9)$$

This equation shows that the ratio of the pole frequencies only depends on the variables Q and F .

Figure 1 below shows the circuit in Middlebrook's example, which emphasizes the relevance of the low-entropy form in engineering applications.

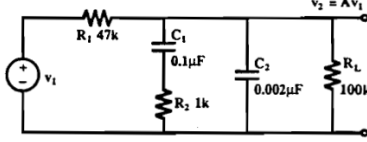


Fig.2. A circuit example to illustrate the improved formula for quadratic roots.

Figure 1: The circuit diagram as shown in Middlebrook's paper.

3 Methodology

For comparison of root finding, we will use the aforementioned quadratic and low-entropy forms as well as two numerical methods. In addition, we will explore the comparison of the cubic formula and the numerical methods.

3.1 Bisection Method

The bisection method is a numerical method that uses the Intermediate Value Theorem to find the roots of a continuous function. It is a relatively simple method that requires only a few steps [1]:

1. Calculate the midpoint, defined as

$$c = \frac{a + b}{2} \quad (10)$$

2. Evaluate $f(c)$

- if $f(c) = 0$, the root is solved
- if $f(a) \cdot f(c) < 0$, then let $b = c$ and evaluate $f(c)$ again
- if $f(a) \cdot f(c) > 0$, then let $a = c$ and evaluate $f(c)$ again

3. Continue applying (10) until $f(c)$ is sufficiently small.

The main flaw with the Bisection Method is that it converges slowly but is guaranteed to converge as long as the initial values of a and b are correct.

3.2 Newton's Method

Newton's method is another numerical method that is used to find roots of a real and continuous function. Newton's method differs from the bisection method as it uses the derivative function to find a root, as shown below [1]:

1. Choose an initial guess, $f(x_0)$ that is close to the root value
2. Calculate $f(x_0)$ and $f'(x_0)$
3. Use the Newton-Raphson Formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (11)$$

4. Continue applying (11) until $|x_{n+1} - x_n|$ is sufficiently small.

3.3 Cubic Formula

The solution for a general cubic polynomial $ax^3 + bx^2 + cx + d = 0$ is significantly more complex than for a general quadratic polynomial. Instead of using the traditional cubic formula, it is often easier to represent the roots using Cardano's formula (which is also implemented in the analysis section) [2]: The roots are given by:

$$x = \sqrt[3]{-\frac{q}{2} + \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}} - \frac{b}{3a} \quad (12)$$

where

$$p = \frac{3ac - b^2}{3a^2}, \quad q = \frac{2b^3 - 9abc + 27a^2d}{27a^3} \quad (13)$$

Looking at (12), one can notice many issues with the formula. Not only is the value of b is concerned, there are more issues with domain restrictions due to multiple square root terms. Due to the length of the formula, the cubic equation will have a greater number of truncation issues than the quadratic formula. The data later in the report will highlight this issue as.

4 Analysis

This report will compare the algebraic formulas and the aforementioned numerical methods using Python 3 on a Windows 11 operating system. The comparison will focus on solving quadratic and cubic polynomials. For quadratic polynomials, we will evaluate the accuracy of the two numerical methods, as well as the quadratic formula and the low entropy method. For cubic polynomials, we will evaluate the accuracy of the two numerical methods and the cubic formula. The analysis will include tables that compare the values of the roots as well as the number of iterations for the two numerical methods.

4.1 Quadratic Case

For the quadratic case, the two tables below depict data for $a = c = 1$ while modifying the value for b .

Method	Small Root	Big Root	Iterations
Quadratic Formula	0.000	5.000e11	N/A
Bisection	2.000e-12	5.001e11	51
Newton's	2.000e-12	5.000e11	3
Low-Entropy	2.000e-12	5.000e11	N/A

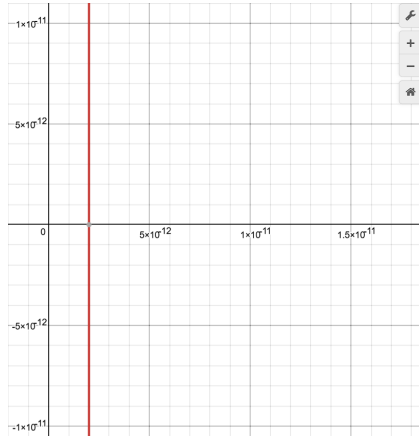
Table 1: Data for $b = -5 \times 10^{11}$

Method	Small Root	Big Root	Iterations
Quadratic Formula	0.000	5.300e11	N/A
Bisection	1.887e-12	5.299e11	51
Newton's	1.887e-12	5.300e11	3
Low-Entropy	1.887e-12	5.300e11	N/A

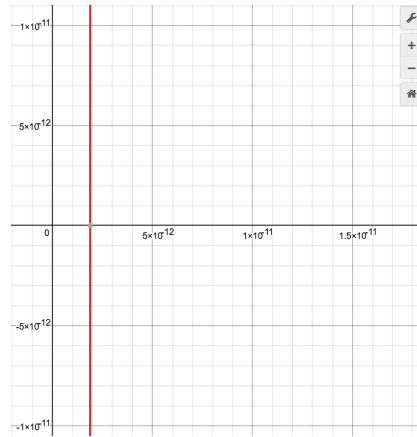
Table 2: Data for $b = -5.3 \times 10^{11}$

As we can see from the tables above, the methods are all in agreement for the larger root. However, the quadratic formula is in disagreement with the other three methods regarding the smaller root. Additionally, note that the bisection method is inefficient in terms of time, since it takes 27 times the number of iterations as Newton's method.

To prove that the small root is not equal to exactly 0, we can plot the equations and notice that the small root has a nonzero value:



The quadratic equation $y = x^2 + (-5 \times 10^{11})x + 1$ graphed with Desmos



The quadratic equation $y = x^2 + (-5.3 \times 10^{11})x + 1$ graphed with Desmos

4.2 Cubic Case

For the cubic case, the two tables below depict the data with $a = c = d = 1$ and changes for various b values. Here is the table below:

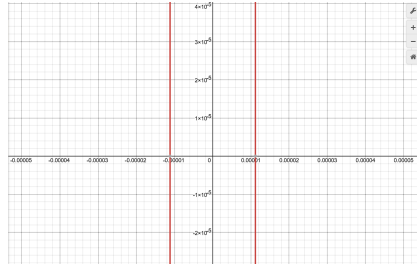
Method	Root 1	Root 2	Root 3	Iterations
Cubic Formula	8.000e9	N/A	N/A	N/A
Bisection	8.000e9	1.118e-05	-1.144e-05	51
Newton's	8.000e9	1.118e-05	-1.144e-05	21

Table 3: Data for $b = -8 \times 10^9$

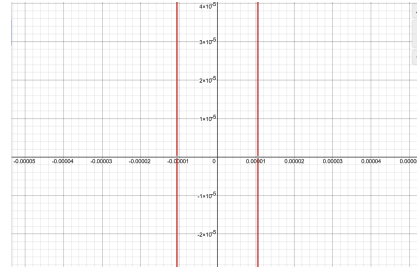
Method	Root 1	Root 2	Root 3	Iterations
Cubic Formula	9.000e9	25.810	-25.810	N/A
Bisection	9.000e9	1.054e-5	-1.049e-5	51
Newton's	9.000e9	1.054e-5	-1.049e-5	21

Table 4: Data for $b = -9 \times 10^9$

For Table 3, the cubic formula was unable to find the two smaller roots, while the numerical methods had no trouble calculating them. This is caused by truncation issues with Python as well as domain restrictions within the cubic formula. As we can see from the graph below, the 2 smaller roots do exist:



The cubic equation $y = x^3 + (-8 \times 10^9)x^2 + x + 1$ graphed with Desmos



The cubic equation $y = x^3 + (-9 \times 10^9)x^2 + x + 1$ graphed with Desmos

5 Conclusion

We observe that the quadratic and cubic formulas become numerically unstable for extreme coefficient values when calculated on a computer. This inaccuracy is caused by floating-point truncation and rounding errors when calculating with a machine. Python and most other programming languages use fixed-precision floating-point numbers that can only store a certain amount of digits. As a result, extremely small nonzero values are rounded to zero, causing a loss of precision when formulas have subtraction present.

In order to mitigate these issues, alternate algebraic forms or numerical methods are used to avoid the subtraction cancellation. These alternative methods provide stable and accurate solutions and may be preferred when numerical accuracy is required.

6 Further Research

While this paper focuses on quadratic and cubic polynomials, further investigation is needed for quartic polynomials. Furthermore, this paper only mentions variations in the value of b while holding the other coefficient values constant. The report covers only a low-entropy form for the quadratic equations but fails to mention if there exist low-entropy forms for either cubic or quartic equations.

References

- [1] Richard L Burden and J Douglas Faires. *Numerical analysis, brooks*. 1997.
- [2] *Cardano's Method*. <https://www.math.ucdavis.edu/~kkreith/tutorials/sample.lesson/cardano.html>. Accessed: 2025-05-04.
- [3] RD Middlebrook. "Methods of design-oriented analysis: The quadratic equation revisited". In: *FRONTIERS IN EDUCATION CONFERENCE*. IEEE. 1992, pp. 95-95.